

17. Which do you think would be a better choice to implement a collection of objects where it must be fast to locate an object by a key: an ordered doubly linked list or an ordered array?
18. Circular lists
 - a. can implement ordered lists more efficiently than doubly linked lists.
 - b. can more efficiently represent loop-like orderings such as turns in a game than doubly linked lists.
 - c. can make finding an item with a matching key less efficient than singly linked lists.
 - d. simplify the representation of lists by eliminating the need for a special value indicating the end of a list.
19. What are the consequences of making the `setNext()` method of a `Link` object public, assuming that it performs only the assignment if the argument is a reference to a `Link` object or `None`?
20. Python generators
 - a. are useful for creating iterators.
 - b. can be used to make classes like circular lists from simpler classes.
 - c. are the only way to represent infinite sequences like the Fibonacci sequence.
 - d. require the use of the `yield` statement and cannot contain loops.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 5-A The `OrderedList` class shown in [Listing 5-13](#) has a `find()` method that steps through the list in increasing order to find the goal. Would it be better to do a binary search instead of a linear one? Why or why not?
- 5-B Imagine that you gave a friend a gift last year, say a nice candle. This year a different friend gives you a nice candle as a gift. You wonder if the exact same candle, not just a similar one, has been returned to you. How would you go about determining whether it's the same one? How does that process relate to the concepts in this chapter?

5-C Imagine a gift exchange among a group of people. Each person will give a small gift to one other person in the group, and the receiver won't know who brought their gift. To set up this exchange, you put the names of all the people in a box, and everyone takes turns drawing a name from the box. If the name they draw is their own, they draw another name from the box and put theirs back in. The name is the person who will receive their gift. This selection process continues until everyone has drawn a name. These are sort of like the links in lists.

Here's how the exchange runs:

- One gift is given each day.
- Each person gives a gift the day after they receive their gift.
- The first person to draw a name gives their gift on day 1.

How long will the gift giving take if there were N uniquely named people? Can you think of any problems that could affect the answer? Can you think of ways to eliminate the problems?

5-D Iterators are great for stepping through data structures to find an item. Would you use one if you want to delete that item? Specifically, if you used one of the iterators shown in [Listing 5-21](#) or [Listing 5-23](#) to find the item to delete, what would be the next step? How efficient would that step be?

5-E Linked lists use memory more efficiently than expandable arrays, but by how much? Imagine an application that starts with an array of 10 elements. It inserts items into the array and doubles the size of the array every time an insertion would go past the end of the current array size. When it doubles the size of the array, it allocates a new segment of memory that is twice as long and copies all the contents of the first array into the beginning half of the second array. How much more memory is used by the expandable arrays if the number of items inserted is 1,000? How about for 1,000,000? Assume that the smaller arrays still take up space even when they are no longer used, and that storing 1 item takes as much memory as storing 1 reference.

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 5.1 Rewrite the `traverse()`, `__str__()`, and `__len__()` methods of the `LinkedList` class shown in [Listing 5-4](#) to use the iterator (created by the generator) shown in [Listing 5-23](#).
- 5.2 Implement a priority queue based on an ordered linked list. The items stored in the queue can be passed to a function that extracts their priority value (or key) as in the `PriorityQueue.py` module in [Chapter 4](#). The remove operation on the priority queue should remove the item with the smallest key.
- 5.3 Implement a deque based on a doubly linked list. (See Programming Project 4.3 in the preceding chapter.) It should include `insertLeft()`, `insertRight()`, `removeLeft()`, `removeRight()`, `peekLeft()`, `peekRight()`, and `isEmpty()` methods.
- 5.4 Make a class for a singly linked circular list that has no end and no beginning like the one shown in [Figure 5-14](#). The only access to the list is a single reference, `__last`, that can point to any link on the list. This reference can move around the list as needed. Your data structure should have methods to check if the list is empty and inspect the first item. It should provide methods for `insertFirst()`, `insertLast()`, `deleteFirst()`, and `search()` (but not `deleteLast()`), ensuring that the list always remains circular. You should provide a `__str__()` method to display the list from first to last (and you need to break the loop before repeating any items). A `step()` method that moves `__last` along to the next link and a `seek()` method that advances it to the next link that matches a particular goal key might come in handy too.
- 5.5 Implement stack and queue classes based on the circular list of Programming Project 5.4. The `Stack` class must have `push()`, `pop()`, and `peek()` methods. The `Queue` class must have `insert()`, `remove()`, and `peek()` methods. This exercise is not too difficult, although you should be careful about maintaining the LIFO and FIFO orders. (Implementing a deque can be harder, unless you make the circular list doubly linked or allow deleting from one end to be $O(N)$.)

- 5.6 Implement another priority queue class as a two-level structure. The top-level list is a list of queues, one for each of the different priority values stored so far. The second level queues hold all the items of the same priority in a way that makes it easy to enforce the queue's first-in, first-out order. This time, the priority of an item is a separate argument to the `insert()` method, not something that is computed by a function on the item being inserted. This allows items to be reprioritized and placed at the end of their new priority queue. The `insert()` method should search the top-level list to find a queue with the matching priority or create a new queue if none of them match. The `remove()` method should take an optional argument, `priority`, which selects a priority queue from which to remove the next item. If `priority` is `None`, then `remove()` should find the highest priority queue that has at least one item. The class should have an iterator to step through all items in all the queues and a second iterator, `priorities()`, to iterate over the priority keys that have at least one item in their queue. Use the full iterator to get the item count for all priorities in the `__len__()` method of the class you define.
- 5.7 Create an iterator with both a `next()` and a `previous()` method to step through the Fibonacci sequence both forward and backward. Because the series is not infinite in both directions, the `previous()` method should raise `StopIteration` instead of returning 0 or a negative number. Calling `previous()` right after `next()` should repeat the same number in the sequence because the iterator has already passed that number. Similarly, calling `next()` right after `previous()` should produce a repeated number. Hint: You can't implement this project with a Python generator because it creates only a single directional iterator.

6. Recursion

In This Chapter

- [Triangular Numbers](#)
- [Factorials](#)
- [Anagrams](#)
- [A Recursive Binary Search](#)
- [The Tower of Hanoi](#)
- [Sorting with mergesort](#)
- [Eliminating Recursion](#)
- [Some Interesting Recursive Applications](#)

Recursion is a programming technique in which a function calls itself. This behavior may sound strange, or even catastrophic. Recursion is, however, one of the most interesting, and one of the most surprisingly effective, techniques in programming. It's been said, "The definition of insanity is doing the same thing over and over again, but expecting different results," so how could a function calling itself ever get a better result? It not only works but also provides a unique conceptual framework for solving many problems.

The concept of recursion appears in the natural world and in art, such as the label used on the early twentieth century Droste Cacao tin shown in [Figure 6-1](#). The image of the tin appears on the tray carried by the nurse. Presumably, that smaller image would contain the entire image of the tin, and that tin's label would contain the entire image of the tin, and so on. This illustration was a popular example of recursion in art, leading to the name *Droste effect*. Mirrors placed facing each other and aligned to be parallel produce infinite recursive images. Many patterns in nature repeat at smaller scales within their structures.



Figure 6-1 *The Droste Effect—recursion in art*

In this chapter we examine numerous examples to show the wide variety of situations to which recursion can be applied in programming. We show you how to calculate triangular numbers and factorials, generate anagrams, perform a recursive binary search, solve the Tower of Hanoi puzzle, and investigate a sorting technique called mergesort. Visualization tools are provided to demonstrate the Tower of Hanoi and mergesort.

We also discuss the strengths and weaknesses of recursion and show how recursive approaches can be transformed into stack-based approaches.

Triangular Numbers

It's said that the Pythagoreans, a band of mathematicians in ancient Greece who worked under Pythagoras (of Pythagorean theorem fame), felt a mystical connection with the series of numbers 1, 3, 6, 10, 15, 21, 28, ... (where the ... means the series continues indefinitely). Can you find the next member of this series?

The n th term in the series is obtained by adding n to the previous term. For example, the second term is found by adding 2 to the first term (which is 1), giving 3. The third term is 3 added to the second term (which is 3) giving 6, and so on. You can also consider the 0th term to be 0, because adding 1 to it gives the first term. The numbers in this series are called **triangular numbers** because they can be visualized as a triangular arrangement of objects, shown as blue squares in [Figure 6-2](#). The length of the shorter (equilateral) sides is n , the term number.

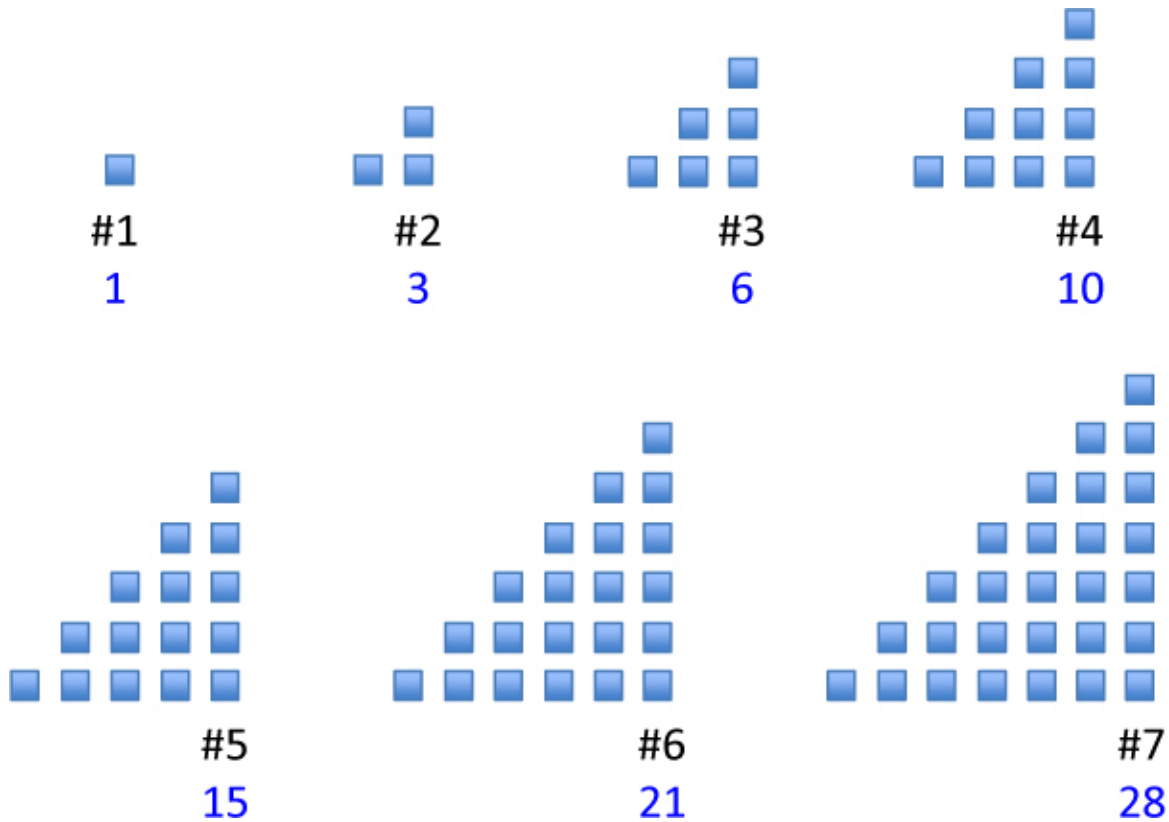


Figure 6-2 *The triangular numbers*

Finding the n th Term Using a Loop

Suppose you wanted to find the value of some arbitrary n th term in the series—say the fourth term (whose value is 10). How would you calculate it? Looking at [Figure 6-3](#), you might decide to determine the value of any term by adding up the number of squares in the vertical columns.

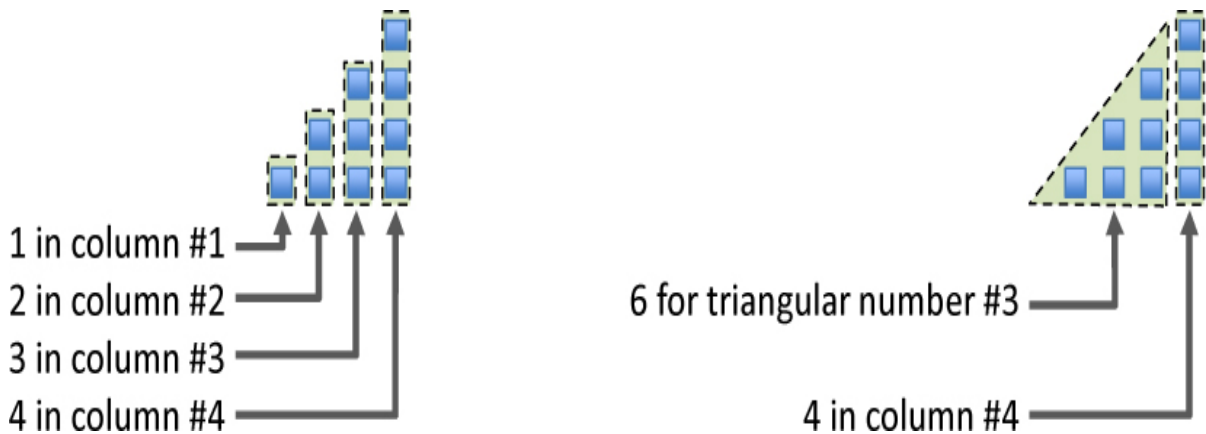


Figure 6-3 *Triangular number as columns*

There are four squares in the fourth column. The value of the fourth term is 4 plus the value of the third term. The third term adds the three squares in its column to the second term, and so on down to column #1 with one square. Adding $4+3+2+1$ gives 10.

The following `triangular_loop()` function uses this column-based technique to find the n th triangular number. It sums all the columns, from a height of n to a height of 1:

```
def triangular_loop(nth): # Get the nth triangular number using a loop
    total = 0             # Keep a total of all the columns
    for n in range(nth, 0, -1): # Start at nth and go back to 1
        total += n         # add n (column height) to total
    return total          # Return the total of all the columns
```

Of course, the values could be added up in any order such as $1+2+3+4$, which would change the range iterator to `range(1, nth + 1)`.

Finding the n th Term Using Recursion

The loop approach is straightforward, but there's another way to look at this problem. The value of the n th term can be thought of as the sum of only two things, instead of a whole series. They are

1. The n th (tallest) column, which has the value n
2. The sum of all the remaining columns

This concept is shown in [Figure 6-4](#).



Figure 6-4 *Triangular number as column plus triangle*

Finding the Remaining Columns

If we knew a method that found the sum of all the preceding columns, we could write a `triangular()` method, which returns the value of the n th triangular number, like this:

```
def triangular(nth):      # Get the nth triangular number (incomplete)
    return (nth +         # Add this column to the preceding
            sum_before(nth)) # column sum
```

But what have we gained here? It looks like writing the `sum_before()` method is just as hard as writing the `triangular()` method in the first place.

Notice in [Figure 6-4](#), however, that the sum of all the preceding columns for term n is the same as the sum of *all* the columns for term $n-1$. Thus, if we knew about a method that summed all the columns up to term n , we could call it with an argument of $n-1$ to find the sum of all the remaining columns for term n :

```
def triangular(nth):      # Get the nth triangular number (incomplete)
    return (nth +         # Add this column to the preceding
            sum_up_to(nth - 1)) # column sum
```

When you think about it, the `sum_up_to()` method does exactly the same thing that the `triangular()` method does: sum all the columns for some number n passed as an argument. So why not use the `triangular()` method itself, instead of some other method? That would look like this:

```
def triangular(nth):      # Get the nth triangular number (incomplete)
    return (nth +         # Otherwise add this column to the preceding
            triangular(nth - 1)) # triangular number
```

The fact that a function can call itself might seem odd, but why shouldn't it be able to? A function call is, among other things, a transfer of control to the start of the function. This transfer of control can take place from within the function as well as from outside. The same is true, of course, for object methods.

Passing the Buck

All these approaches may seem like passing the buck. Someone asks you to find the 9th triangular number. You know this is 9 plus the 8th triangular

number, so you call Harry and ask him to find the 8th triangular number. When you hear back from him, you can add 9 to whatever he tells you, and that will be the answer.

Harry knows the 8th triangular number is 8 plus the 7th triangular number, so he calls Sally and asks her to find the 7th triangular number. This process continues with each person passing the buck to another one.

Where does this buck-passing end? Someone at some point must be able to figure out an answer that doesn't involve asking another person to help. If this didn't happen, there would be an infinite chain of people asking other people questions—a sort of arithmetic Ponzi scheme that would never end. In the case of `triangular()`, this would mean the function calls itself over and over in an infinite series that would eventually crash the program.

The Buck Stops Here

To prevent an infinite regress, the person who is asked to find the first triangular number of the series, when n is 1 (or 0), must know, without asking anyone else, that the answer is 1 (or 0). There are no smaller numbers to ask anyone about, there's nothing left to add to anything else, so the “buck” stops there. In fact, we certainly don't want anyone to make a mistake and ask about negative numbers, which might throw off the total count. We can express this by adding a condition to the `triangular()` method, as shown in [Listing 6-1](#).

Listing 6-1 *The Recursive `triangular()` Function*

```
def triangular(nth):    # Get the nth triangular number recursively
    if nth < 1: return 0 # For anything less than 1, it's 0
    return (nth +       # Otherwise add this column to the preceding
            triangular(nth - 1)) # triangular number
```

The condition that leads to a recursive method returning without making another recursive call is referred to as the **base case**. It's critical that every recursive method have a base case to prevent infinite recursion and the consequent demise of the program.

Using the recursive definition described earlier, you can ask for large triangular numbers like this:

```
>>> triangular(100)
5050
```

```
>>> triangular(200)
20100
>>> triangular(500)
125250
```

What's Really Happening?

Let's modify the `triangular()` function to provide an insight into what's happening when it executes. We'll insert some print statements to keep track of the arguments and return values:

```
def show_triangular(nth): # Print the recursive execution steps of
    print('Computing triangular number #', nth) # computing the nth
    if nth < 1: # triangular number. Base case
        print('Base case. Returning 0') # Print the return information
        return 0
    value = nth + show_triangular(nth - 1) # Non-base case, get value
    print('Returning', value, 'for #', nth) # Print the return info
    return value
```

The print statements show the entry and exit information for each call to `show_triangular()`. Here's the result of calling `show_triangular(5)`:

```
>>> show_triangular(5)
Computing triangular number # 5
Computing triangular number # 4
Computing triangular number # 3
Computing triangular number # 2
Computing triangular number # 1
Computing triangular number # 0
Base case. Returning 0
Returning 1 for # 1
Returning 3 for # 2
Returning 6 for # 3
Returning 10 for # 4
Returning 15 for # 5
15
```

Each time the `show_triangular()` function calls itself, its argument is reduced by 1. The function plunges down calling itself again and again until its argument is reduced to 0. Then it returns. This process triggers an entire series of returns. The control passes back up out of all the versions that were waiting for a value. Each time the function returns, the caller adds its value of `nth` to the return value of the function it called.

The return values reconstruct the series of triangular numbers, until the answer is returned to the Python interpreter. [Figure 6-5](#) shows how each invocation of the `triangular()` function can be imagined as being "inside" the previous one.



Figure 6-5 *Execution of the recursive `triangular(4)` function*

Notice that, just before the innermost version returns a 0, there are actually five different incarnations of `triangular()` in existence at the same time. The outer one was passed the argument 4; the inner one was passed the argument 0. The top and bottom edges of each invocation correspond to the print statements in `show_triangular()`.

Characteristics of Recursive Routines

Although it's short, the `triangular()` function possesses the key features common to all recursive routines:

- It calls itself.
- When it calls itself, it does so to solve a smaller version of the same problem, also called a subproblem.
- There's some version of the problem that is simple enough that the routine can solve it, and return, without calling itself.

In each successive call of a recursive method to itself, the argument becomes smaller (or perhaps a range described by multiple arguments becomes smaller), reflecting the fact that the problem has become "smaller" or easier. Here we explore how data structures holding fewer data elements can be the smaller

problem. When the argument or range reaches a certain minimum size, a condition triggers and the method returns without calling itself.

Is Recursion Efficient?

Calling a function or method involves certain overhead. Control must be transferred from the location of the call to the beginning of the function. In addition, the arguments to the function and the address to which the function should return must be pushed onto an internal stack so that the function can access the argument values and know where to return.

In the case of the `triangular()` function, it's probable that, as a result of this overhead, the `while` loop approach executes more quickly than the recursive approach. The penalty may not be significant, but if there are a large number of function calls as a result of a recursive function, it might be desirable to eliminate the recursion. We discuss this issue more at the end of this chapter.

Another inefficiency is that memory is used to store all the intermediate arguments and return values on the system's internal stack. This behavior may cause problems if there is a large amount of data, leading to a stack overflow.

These efficiency concerns are especially relevant to triangular numbers and other numeric computations. There are usually mathematical simplifications that can avoid loops or recursion. As you might have guessed, the number of little squares in the triangle is about equal to the area of the triangle, which is half the area of the square formed by putting two of the triangles together, sharing their longest edge. For a square of width n , its area is n^2 , so the area of the triangle would be $n^2/2$. The exact formula for the sum of all the integers from 0 to a positive integer, n , is

$$\frac{n(n + 1)}{2}$$

This formula shows up again and again in the study of data structures. The most efficient way to calculate it would be to implement the computation of this formula because the addition, multiplication, and division are all $O(1)$ operations.

Recursion is usually used because it simplifies a problem *conceptually*, not because it's inherently more efficient.

Mathematical Induction

For those programmers with an interest in math, it's important to see the relationship between recursion and induction. **Mathematical induction** is a way of defining something in terms of itself. (The term is also used to describe a related approach to proving theorems, which is also a technique for proving the correctness of programs.) Using induction, you could define the triangular numbers mathematically by saying

$$\text{tri}(n) = 0 \quad \text{if } n \leq 0$$

$$\text{tri}(n) = n + \text{tri}(n-1) \quad \text{if } n \geq 1$$

Defining something in terms of itself may seem circular, but in fact it's perfectly valid (provided there's a base case).

Factorials

Factorials are similar in concept to triangular numbers, except that multiplication is used instead of addition. The triangular number corresponding to n is found by adding n to the triangular number of $n-1$, while the factorial of n is found by multiplying n by the factorial of $n-1$. In other words, the fifth triangular number is $5+4+3+2+1$, while the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1$, which equals 120.

Factorials are useful for finding the number of possible orderings or combinations of things. Think of a race between three people. How many different ways can the race end assuming no ties occur? The first person can finish in first, second, or third position. The second person can finish in any position other than the first, so two positions. The third person takes the final position. So, there are $3 \times 2 \times 1$, which equals 6, orderings. That same logic extends to higher numbers. [Table 6-1](#) shows the factorials of the first 10 numbers.

Table 6-1 *Table Factorials*

Number	Calculation	Factorial
0	by definition	1
1	1×1	1
2	2×1	2
3	3×2	6
4	4×6	24
5	5×24	120
6	6×120	720
7	7×720	5,040
8	$8 \times 5,040$	40,320
9	$9 \times 40,320$	362,880

The factorial of 0 is defined to be 1. If you think about the example of the orderings of a group of racers, there is exactly 1 ordering of an empty group of racers. Factorial numbers grow large very rapidly, as you can see. In math, the factorial of 5 is written with an exclamation mark, $5!$, perhaps as a way to remind us how big they are.

A recursive method similar to `triangular()` can be used to calculate factorials. It looks like this:

```
def factorial(n):           # Get factorial of n
    if n < 1: return 1      # It's 1 for anything < 1
    return (n *            # Otherwise, multiply n
            factorial(n - 1)) # by preceding factorial
```

There are only a few differences between `factorial()` and `triangular()`. First, we changed the variable to `n` from `nth` because factorial is treated as a function rather than a sequence. Second, `factorial()` uses a `*` instead of a `+` in the expression

```
n * factorial(n - 1)
```

Third, the base condition returns 1, not 0. It's possible to eliminate one or more recursive calls by adding more base cases for $n = 1$, $n = 2$, and so on, but doing

so only saves a little work. Again, the purpose of recursion is its conceptual simplicity.

Here are some samples of what the function produces. [Figure 6-6](#) shows how the recursion creates a similar set of invocations of the function to produce the last value.

```
>>> factorial(9)
362880
>>> factorial(40)
815915283247897734345611269596115894272000000000
>>> factorial(4)
24
```



Figure 6-6 *Execution of the recursive `factorial(4)` function*

Various other numeric entities lend themselves to calculation using recursion in a similar way, such as finding the greatest common denominator of two numbers (which is used to reduce a fraction to lowest terms), raising a number to a power, and so on. The factorial function is somewhat more interesting because it can't be easily simplified into a formula using other math primitives that can be computed in constant time. Even though this calculation is interesting for demonstrating recursion, it probably wouldn't be used in practice because a loop-based approach is more efficient. Recursion can be used to prototype an algorithm, which is later converted to a more efficient implementation.

Anagrams

Here's a different kind of situation in which recursion provides a neat solution to a problem. A **permutation** is an arrangement of things in a definite order (sometimes called an *ordering*). Suppose you want to list all the **anagrams** of a specified word—that is, all possible permutations that can be made from the letters of the original word (whether they make a real word or not). We call this *anagramming* a word. Anagramming `cat`, for example, would produce

```
cat
cta
atc
act
tca
tac
```

Try anagramming some words yourself. Because this process is analogous to ordering of the racers, the number of possibilities is the factorial of the number of letters. For 3 letters, there are 6 possible words (permutations); for 4 letters there are 24 words; for 5 letters, 120; and so on. These numbers assume that all letters are distinct; if there are multiple instances of the same letter, there will be fewer distinct words.

How would you write a program to find all the anagrams of a word? Think about it with this new technique, recursion. Here's where the simplifying concept helps. adding n to the triangular number of $n-1$ Assume you already have a function for anagramming a string of n letters. Now all you need is a way to handle adding another letter. If there are $n+1$ letters in the original word, where can that extra letter go in each of the short anagrams of length n ? Well, it could go in between any of the other letters or at the beginning or at the end. Is that all the anagrams with $n+1$ letters?

Think about the example with `cat`. The two anagrams of `at` are `at` and `ta`. If you place `c` between the two characters, you get `act` and `tca`. If you add `c` to the beginning, you get `cat` and `cta`. Adding `c` to the end produces `atc` and `tac`. Altogether you have

```
act, tca, cat, cta, atc, tac
```

You have found all six orderings and haven't introduced any duplicates (as long as there were no duplicates in the list of anagrams at the start). So this looks correct for producing all the possible anagrams when adding a single character to a set of distinct anagrams. All that's left to do is define the base case (or cases).

If a string has one character in it, then there's only one ordering, the string itself. What about an empty string? Well, you should still return the sole ordering of that too, the empty string itself. That corresponds to `factorial(0)` returning 1 as discussed previously. So, you can set up a recursive `anagrams()` function as follows:

```
def anagrams(word):
    if len(word) <= 1:
        return [word]
    result = []
    for part in anagrams(word[1:]):
        for i in range(len(part) + 1):
            result.append(
                part[:i] +
                word[0] +
                part[i:])
    return result
```

The function starts with the base case. Short words of 0 or 1 character are the only possible anagram, so the function immediately returns a list of the `word` itself. If the word is longer than 1 character, it gets all the anagrams of the last part of `word` (all but the first character) by using the Python slice operator (`word[1:]`). It then loops over each one of those shorter anagrams, storing it in the `part` variable. Before entering the loop, it initializes a `result` list to be the empty list.

Within each `part`, there are `n`, or `len(part)`, characters. The first character of the `word` to be anagrammed can go between any of them or at the beginning or the end. That's the same as inserting the first character before any character of `part` plus inserting it at the `n+1` position. The function uses another loop to set `i` to each of those insertion indices, 0 up to `n`, inclusive. The body of the inner loop appends a new `n+1` character anagram to the `result` list. (In this case, Python's `list` type is being treated as a linked list rather than an array.) The longer anagram starts with the (possibly empty) slice of the shorter anagram, `part`, up to `i`, then the first letter of the `word`, and then the (possibly empty) remaining slice of the shorter anagram starting at `i`.

Testing this implementation, you get

```
>>> anagrams('')
['']
>>> anagrams('c')
['c']
```

```
>>> anagrams('cat')
['cat', 'act', 'atc', 'cta', 'tca', 'tac']
```

The invocations of the anagram function on "cat" are shown in [Figure 6-7](#). The innermost invocation is the base case, which returns the list of one string, the last letter, "t". The invocation surrounding that innermost one inserts "a" into the "t" string at the two possible locations, the beginning and the end, to produce ["at", "ta"]. The outermost invocation inserts "c" into each of the two strings at one of three possible locations. The combination of two strings and three possible locations produces six anagrams.



Figure 6-7 *Anagramming a word*

Anagramming short words is fine. For example:

```
>>> anagrams('tire')
['tire', 'itre', 'irte', 'iret', 'trie', 'rtie', 'rite', 'riet',
'trei',
'rtet', 'reti', 'reit', 'tier', 'iter', 'ietr', 'iert', 'teir',
'etir',
'eitr', 'eirt', 'teri', 'etri', 'erti', 'erit']
```

Anagramming longer words, however, is likely to become more of a nuisance. The factorial of 6 is 720, and generating such long sequences may produce more words than you want to know about.

A Recursive Binary Search

Remember the binary search we discussed in [Chapter 2](#), "Arrays"? The search finds the index to a cell with a matching key in an ordered array using the

fewest number of comparisons. The solution kept dividing the array in half, seeing which half contained the desired cell, dividing that half in half again, and so on. Here's the `OrderedRecordArray.find()` method:

```
def find(self, key):
    lo = 0
    hi = self.__nItems-1
    while lo <= hi:
        mid = (lo + hi) // 2

        if self.__key(self.__a[mid]) == key:
            return mid

        elif self.__key(self.__a[mid]) < key:
            lo = mid + 1

        else:
            hi = mid - 1

    return lo
```

Find index at or just below key
in ordered list
Look between lo and hi
Select the midpoint
Did we find it?
Return location of item
Is key in upper half?
Yes, raise the lo boundary
No, but could be in lower half
Item not found, return insertion point instead

You might want to reread the section on binary searches in ordered arrays in [Chapter 2](#), which describes how this method works or review the algorithm visualization.

You can transform this loop-based method into a recursive method quite easily. The loop-based method changes either `lo` or `hi` to specify a new range on each iteration. Each time through the loop it divides the range (roughly) in half.

Recursion Replaces the Loop

In the recursive approach, instead of changing `lo` or `hi` inside a loop, you call `find()` again with the new values of `lo` or `hi` as arguments. The loop disappears, and its place is taken by the recursive calls. Here's how that looks:

```
class OrderedRecordArray(object):
    ... # other definitions as shown in Chapter 2 ...

    def find(self, key,
             lo = 0,
             hi = None):
        if hi is None:
            hi = self.__nItems - 1
        if lo > hi:
            return lo
```

Find index at or just below key
in ordered list between lo
and hi using recursion
If hi was not provided,
use upper bound of array
If range is empty,
return lo for base case

```

mid = (lo + hi) // 2          # Select the midpoint
if self.__key(self.__a[mid]) == key: # Did we find it?
    return mid                # Return location of item

if self.__key(self.__a[mid]) < key: # Is key in upper half?
    return self.find(            # then recursively search
        key, mid + 1, hi)       # in upper half
else:                           # Otherwise, it must be in
    return self.find(           # lower half so recursively
        key, lo, mid - 1)       # search below mid

```

This recursive version defines default values for the `lo` and `hi` parameters to `find()`. That allows callers to call the method without specifying the range of indices to search. The first `if` statement fills in the value of `hi` as the last valid index if the caller did not provide a value (or passed `None` as an argument). The default value for `lo` is a constant, 0, so it can be specified in the parameter definition.

Like the other recursive methods, the next test is for the base case. The second `if` statement looks for an empty range of indices where `lo > hi`. That will happen for empty arrays or when the key is not found among the items in the array. In this case, `find()` returns `lo` because an item with that key would be inserted at `lo` to maintain the ordering. Why not check if `lo == hi` as the base case? If you did so, then you would still need to compare the key being sought with the key of the item at `lo`. Depending on their relationship, the value to return could be `lo`, `lo - 1`, or `lo + 1`. It's easier to skip those checks and let the recursion proceed until `lo > hi`. We show each of those cases in the rest of the routine.

The next statement sets `mid` to be the midpoint of the range between `lo` and `hi`. The third `if` statement compares the key at that index of the array with the `key` being sought. If they are equal, the method can return `mid` as the result; no more recursion is needed. If there were previous recursive calls to get to this point, they all will pass the `mid` value as the result back to their callers.

After the third `if` statement compares the search `key` with the one at `mid` and fails to find a match, the only thing left to try is searching the upper or lower search ranges. By checking whether the search `key` is above or below the key at `mid`, the method can determine the proper range and search it using the recursive call to `self.find()`. These two calls handle the other two conditions related to `lo == hi == mid`. The recursive call goes to `lo + 1` if the key at `mid` is less than the search `key`. The `lo - 1` case is handled by reducing `hi` to `mid -`

1, which is only a possible return value if there were indices below `mid` to be searched.

Here's a small test of the recursive `find()` method. In this example, the program places a small set of integers in an array and tries finding some that are present and some that are not.

```
arr = OrderedRecordArray(10)
for item in [3, 27, 14, 10, 88, 41, 67, 51, 95]:
    arr.insert(item)

print("Array containing", len(arr), "items:\n", arr)

for goal in [0, 10, 11, 99]:
    print("find(", goal, ") returns", arr.find(goal))
```

The output from this test is

```
Array containing 9 items:
[3, 10, 14, 27, 41, 51, 67, 88, 95]
find( 0 ) returns 0
find( 10 ) returns 1
find( 11 ) returns 2
find( 99 ) returns 9
```

Searching for 0 returns 0 because the value would go in the first cell of the array if inserted, so it could be in front of 3. The call to `find(10)` returns 1, which is the index of the cell holding 10. The next call to `find(11)` shows it returning an index one before the next larger integer in the array, and the last one is beyond the current end of the of the array because it is bigger than all the items.

[Figure 6-8](#) shows the recursive calls needed to find the index for 11 in the preceding array. In the initial call, `lo` and `hi` are not specified, so their defaults (shown in green inside the parentheses) are used. The internal recursive calls don't use the defaults.

Begin arr.find(11)

arr =

0	1	2	3	4	5	6	7	8	9
3	10	14	27	41	51	67	88	95	?

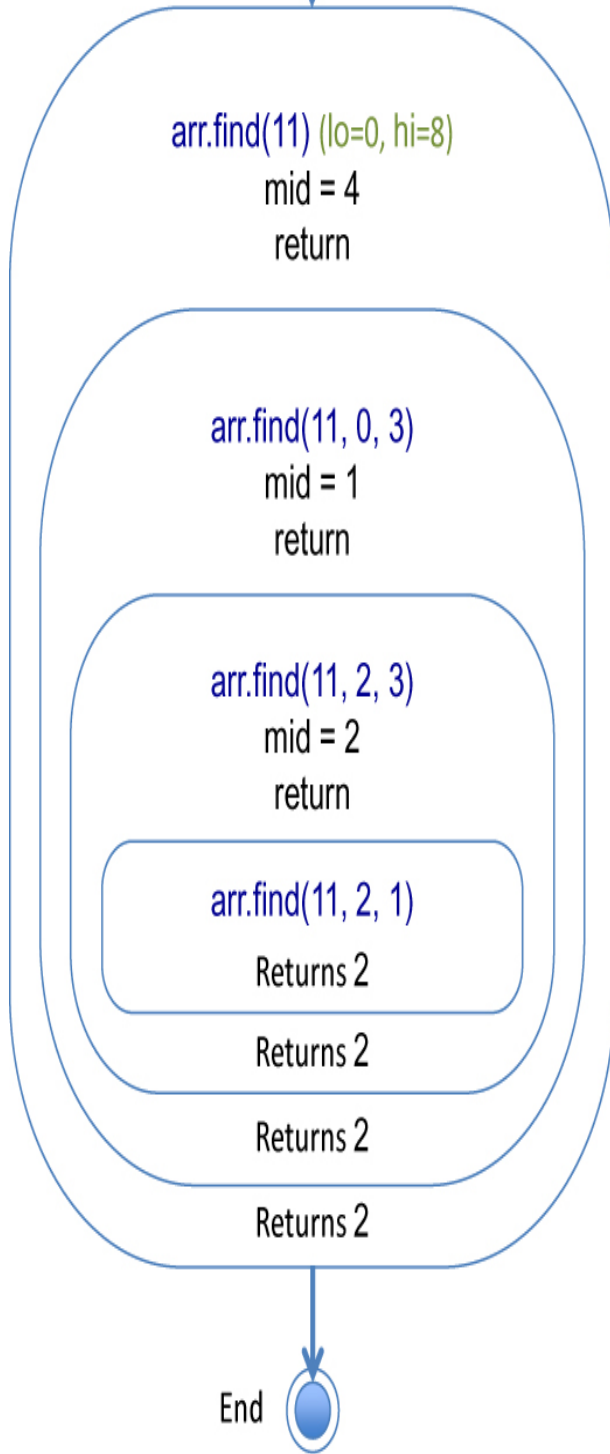


Figure 6-8 *Performing recursive binary search*

The range of indices to search is initially all 9. In the first recursive call, the range shrinks to 4, indices 0 through 3. The second recursive call reduces the range to 2, indices 2 through 3. The last recursive call reduces the range to 0 after finding that the key at index 2, 14, is higher than the goal key of 11. Because the range is empty, the base case causes 2 to be returned up the chain of recursive calls.

The recursive binary search has the same Big O efficiency as the nonrecursive version: $O(\log N)$. It is more elegant but may be slightly slower because function calls and returns can take more time than updating loop variables.

Divide-and-Conquer Algorithms

The recursive binary search is an example of the *divide-and-conquer* approach. You divide the big problem into two smaller problems and solve each one separately. The solution to each smaller problem is the same: you divide it into two even smaller problems and solve them. The process continues until you get to the base case, which can be solved easily, with no further division.

The divide-and-conquer approach is commonly used with recursion, although, as you saw in the binary search in [Chapter 2](#), you can also use a nonrecursive approach. It usually involves a method that contains two recursive calls to itself, one for each half of the problem. In the binary search, there are two such calls, but only one of them is actually executed. (Which one depends on the value of the keys.) The mergesort, which is described later in this chapter, actually executes both recursive calls (to sort two halves of an array).

The Tower of Hanoi

The Tower of Hanoi is a historic puzzle consisting of several disks placed on three spindles or columns, as shown in [Figure 6-9](#). The puzzle is also known as the Tower of Brahma or Lucas's Tower for Édouard Lucas, its inventor. Lucas was a mathematician who studied the Fibonacci sequence and other recursively defined sequences. The goal of the puzzle is to move all of the disks from one spindle to another, following a set of specific rules.

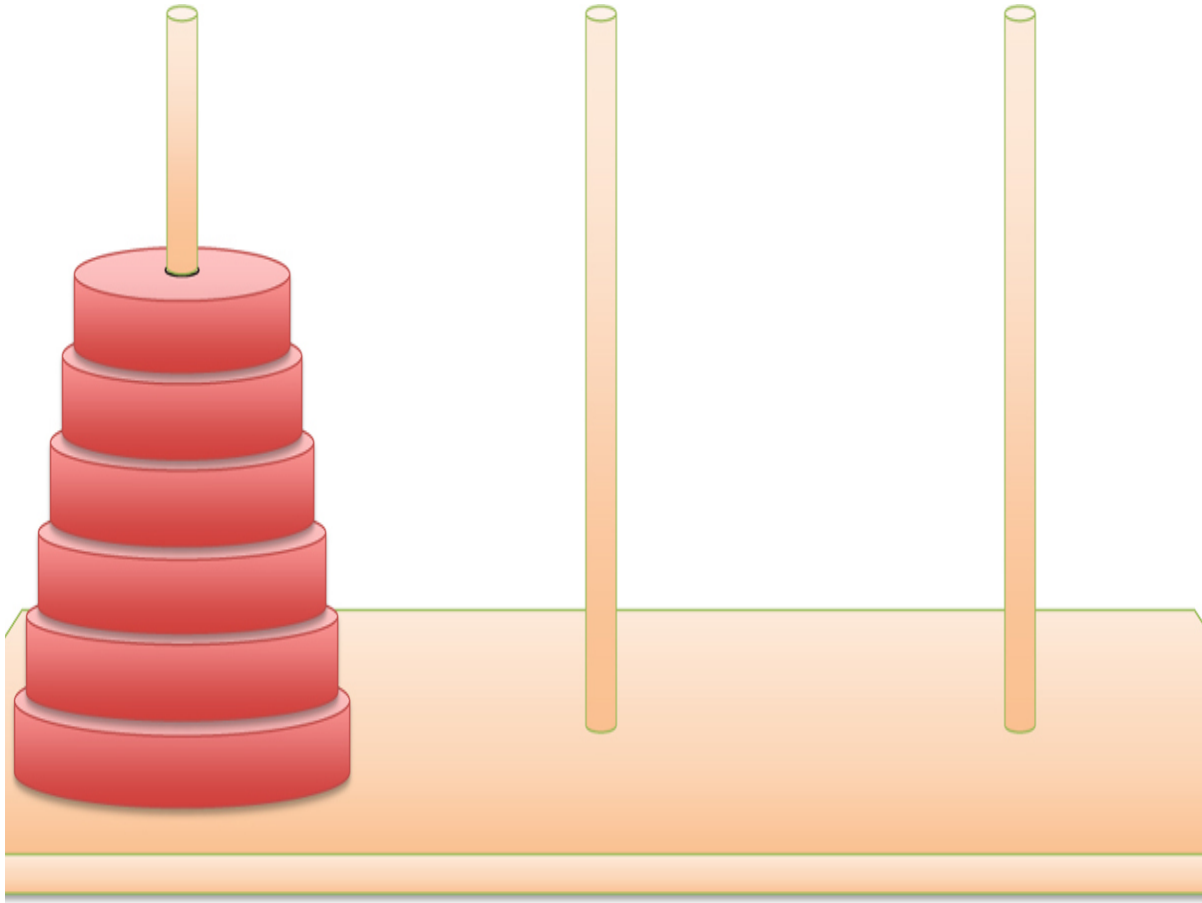


Figure 6-9 *The Tower of Hanoi puzzle*

The disks all have different diameters. Each one has a hole in the middle that fits over the spindles. All the disks start out on one spindle, stacked in order of diameter, which gives the appearance of a tower. The object of the puzzle is to transfer all the disks from the starting spindle, say the one on the left, to another spindle, say the one on the right. Only one disk can be moved at a time, and no disk may be placed on a disk that's smaller than itself.

The legend that goes along with the puzzle is that in a distant temple, a group of monks labor day and night to transfer 64 golden disks from one of three diamond-studded towers to another. When they are finished, the world will end. If that alarms you, wait until you see how long it takes to solve the puzzle with 64 disks.

The Tower of Hanoi Visualization Tool

Start up the TowerOfHanoi Visualization tool. Create a new three-disk puzzle by typing 3 in the text entry box and selecting New. You can attempt to solve the puzzle yourself by using the mouse to drag the topmost disk to another tower. The star by the spindle on the right indicates the goal tower. [Figure 6-10](#) shows how the towers look after several moves have been made.



Figure 6-10 *The Tower Of Hanoi Visualization tool*

There are two ways to use the visualization tool:

- You can attempt to solve the puzzle manually, by dragging the disks from tower to tower.
- You can select the Solve button and watch the algorithm solve the puzzle with no intervention on your part. The disks zip back and forth between the posts. Using the Pause/Play button, you can stop and resume the animation to examine what happens at each step.

To restart the puzzle, type in the number of disks you want to use, from 1 to 6, and select New. The specified number of disks will be arranged on the left spindle. You can drag the top disk to either of the other two spindles. If you pick up the next larger disk, the tool will allow you to place it only on a spindle whose topmost disk has a larger diameter. If you release it away from a spindle that can accept it, the disk returns to where you picked it up. When all the disks form a tower on leftmost spindle, the Solve button is enabled. If you stop the automated solver, you can resume manually solving the puzzle.

Try solving the puzzle manually with a small number of disks, say three or four. Work up to higher numbers. The tool gives you the opportunity to learn

intuitively how the problem is solved.

Moving Pyramids

Let's call the arrangement of disks on a spindle a "pyramid" because they are ordered by diameter. You could call them a "stack," but that would be somewhat confusing with the stack data structure you've studied (although that data structure will be useful to model the disks). Using this terminology, the goal of the puzzle is to move the pyramid from the left spindle to the right. For convenience, the spindles can be labeled L, M, and R for left, middle, and right.

How do you solve the puzzle? Let's start with the easy cases. If there's only one disk, the solution is trivial. Move the one disk from spindle L to spindle R. In fact, even easier, if there are no disks, then there is nothing to move and the puzzle is solved. Then what about two disks? That's pretty easy, too. Move disk 1 from L to M, move disk 2 from L to R, and then move disk 1 from M to R. These moves are shown in the four panels of [Figure 6-11](#).



Figure 6-11 *Solution to the Tower of Hanoi with two disks*

How do you solve problems with more disks? You've seen the base cases; can recursion help with this? You might be surprised to know that you've already figured out all the steps of the recursive algorithm by enumerating the cases up to three disks.

To see how to get to a recursive solution, remember that you need to make smaller versions of the same problem. The two strategies you've seen are dividing the problem in half or reducing the number of things by one. Dividing

it in half doesn't seem to fit, so let's look at reducing the number of disks by one. In particular, let's look at the case of three disks.

If you perform the same steps that you took to solve the two-disk puzzle, then the three-disk puzzle would end up with disk 3 still on the left spindle and disks 1 and 2 on the right spindle, as shown in [Figure 6-12](#). From here, it's easy to move disk 3 onto the middle spindle, but not easy to move it to the right-hand one.

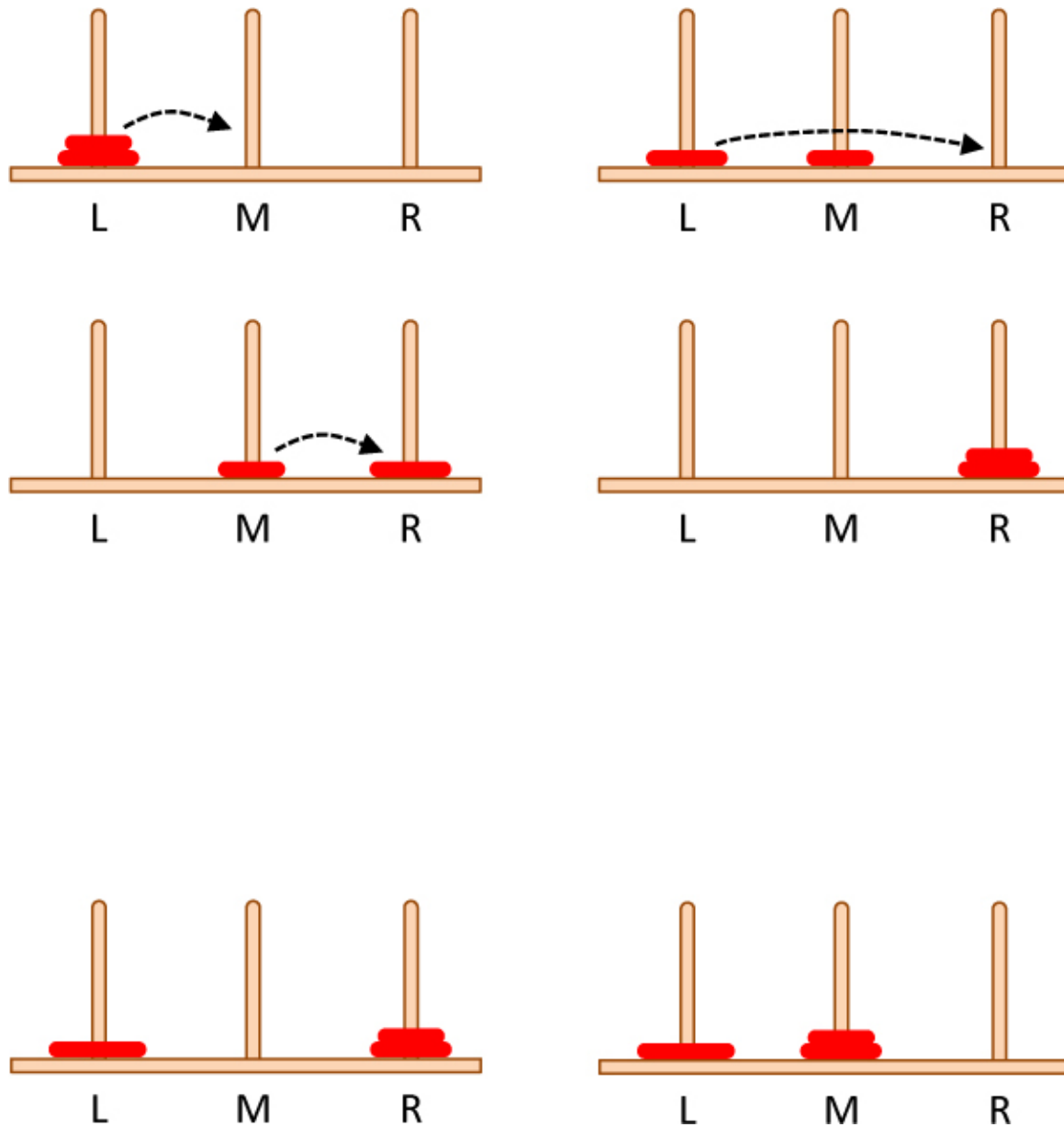


Figure 6-12 *A Tower of Hanoi with three disks after moving two of them*

If you changed the first moves to swap the middle and right spindles, then disks 1 and 2 would end up on the middle spindle and leave the right one open, as shown in [Figure 6-13](#).



Figure 6-13 *A Tower of Hanoi after moving two disks to the middle*

With the right spindle empty, you can easily move disk 3 from the left to the right. Now all that's left is moving the two disks on the middle spindle over to the right. And that's a problem you've already solved. This example provides the basic outline for the solution of the three-disk problem, namely:

1. Move the top two disks to the middle spindle (using the right one as the spare).
2. Move disk 3 to the right spindle.
3. Move the top two disks on the middle spindle to the right spindle (using the left one as the spare).

Steps 1 and 3 are solutions that have the same form as the three moves shown in [Figure 6-11](#). They are solutions to the two-disk problem, but with different starting and ending positions. Now it's time to recognize that you've solved the three-disk problem by reducing it to two calls to the two-disk solution plus one disk move in between. If you rephrased the preceding outline to solve an N disk problem, it would be

1. Solve the $N-1$ disk problem by moving the $N-1$ disk pyramid from the start spindle to the nongoal spindle (using the goal spindle as a spare).
2. Move the N th disk to the goal spindle.

3. Solve the $N-1$ disk problem by moving the $N-1$ disk pyramid from step 1 on to the goal spindle (using the starting spindle as a spare).

Steps 1 and 3 look like recursive calls to the same solution routine. That solution routine would need to know how many disks to move and the role of each of the three spindles: start spindle, goal spindle, and spare spindle. In the various recursive calls, the roles of the spindles change. If you think about the difference between [Figure 6-12](#) and [Figure 6-13](#), they perform the same solution but with the swapped roles of the right two spindles.

Is that it? Have you figured out all the steps? What about all the other, bigger disks? Wouldn't they prevent the solution from working because they were in the way of one of the steps?

If you think about it, it doesn't matter that there are more disks in the puzzle because they are all larger than the topmost disks. The outlined solution applies only to the top N disks, and disks $N+1$, $N+2$, and so on all must be larger. For example, applying the two-disk solution to move from the left to right spindles works equally well in all the situations shown in [Figure 6-14](#), which has five disks. You count on that fact in the outlined solution. In fact, you hope the situation looks like the rightmost panel of [Figure 6-14](#) at some point during the $N = 5$ puzzle. If all five disks started on the left spindle, performing all the outline steps for $N = 5$ and $N = 4$ and then steps 1 and 2 when $N = 3$ should leave disk 3 on the right (goal) spindle with disks 4 and 5 underneath it. All that remains is performing step 3 on the remaining $N-1$ (two) disks.

Begin arr.find(11)

arr =

0	1	2	3	4	5	6	7	8	9
3	10	14	27	41	51	67	88	95	?

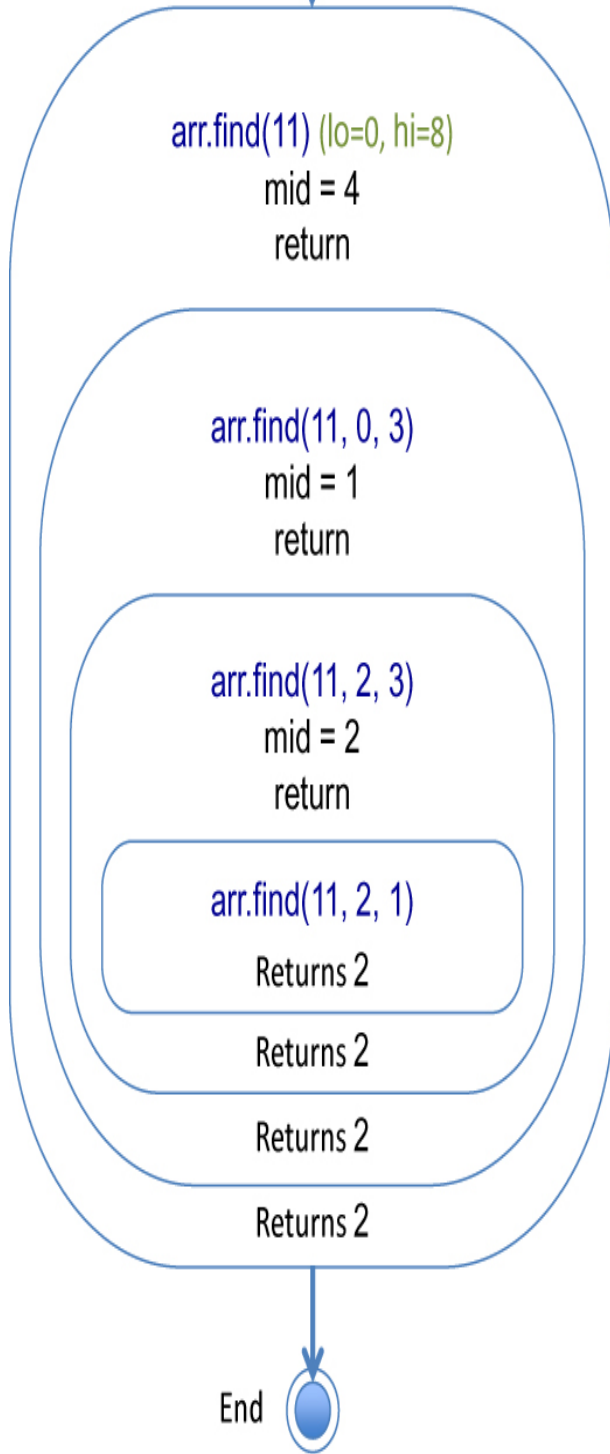


Figure 6-14 Possible states in a five-disk Tower of Hanoi

It still seems as though the algorithm might ask to move a larger disk on top of a smaller one after all the swapping of roles of the spindles. How can you tell that it won't try to move a disk from the right spindle to the left in the rightmost condition show in [Figure 6-14](#)? There are two ways: write a program to test it and use mathematical induction to prove it. We tackle that program next.

The Recursive Implementation

Before diving into the code, let's look at what base cases are needed. As we mentioned, if there are no disks, no moves are needed, and the solution is done. That's definitely a base case. Do you need a base case for a single disk? You can check by seeing what happens if you apply the outline recursive solution to the case where N is 1. Step 1 is to solve the movement of the $N-1$ disk problem. Because $N-1$ is 0, you know that nothing will be moved. That's followed by moving the N th disk to the goal spindle, which means the one disk is moved to the final position. Then step 3 also does nothing (sometimes called a "no-op"), because $N-1$ is 0. Hence, there doesn't seem to be a need to write anything special for the case where N is 1. You only need the base case for 0 and the recursive outline.

[Listing 6-2](#) shows the first part of a class, `TowerOfHanoi`, that solves the puzzle for any number of disks. Each instance of the puzzle will have a specific number of disks that is provided to the constructor. The puzzle needs to keep track of what disks are on what spindles. For that, you could use the `SimpleStack` class that was implemented in [Chapter 4](#), "Stacks and Queues." A stack is perfect for modeling each spindle because the only allowed disk movements involve the top of the stack/spindle. You create a stack for each of the three spindles and push integers on it to represent the diameters of the disks. The diameters can be the numbers from 1 up to N , the number of disks. The bigger numbers are the larger diameter disks.

Listing 6-2 The `TowerOfHanoi.py` Module—Puzzle Object

```
from SimpleStack import *

class TowerOfHanoi(object):    # Model the tower on 3 spindles using
                                # 3 stacks
```



```

def __init__(self, nDisks=3): # Constructor w/ starting number of
    self.__stacks = [None] * 3 # Stacks of disks
    self.__labels = ['L', 'M', 'R'] # Labels for stacks/spindles
    self.__nDisks = nDisks # Total number of disks
    self.reset()

def reset(self): # Initialize state of puzzle
    for spindle in range(3): # Set up each of 3 spindles
        self.__stacks[spindle] = Stack( # Start w/ empty stack
            self.__nDisks) # that can hold all the disks
        if spindle == 0: # On the first spindle,
            for disk in range( # push the disks on the stack
                self.__nDisks, 0, -1): # in descending order of size
                self.__stacks[spindle].push(disk)

def label(self, spindle): # Get the label of spindle
    return self.__labels[spindle]

def height(self, spindle): # Get the number of disks on a spindle
    return len(self.__stacks[spindle])

def topDisk(self, spindle): # Get top disk number on a spindle or
    if not self.__stacks[spindle].isEmpty(): # None if no disks
        return self.__stacks[spindle].peek() # Peek at top disk

def __str__(self): # Show puzzle state as a string
    result = "" # Start with empty string
    for spindle in range(3): # Loop over spindles
        if len(result) > 0: # After first spindle,
            result += "\n" # separate stacks on new lines
        result += (
            self.label(spindle) + ': ' + # Add spindle label
            str(self.__stacks[spindle])) # and spindle contents
    return result

```

The constructor for `TowerOfHanoi` initializes the private class attributes. The stacks modeling the spindle contents go in a three-element array called `__stacks`. A separate three-element array, `__labels`, holds the names for the spindles—L, M, and R—as in the earlier figures. The total number of disks is stored in `__nDisks`. A default value of 3 is provided for `nDisks`.

The constructor uses a separate method to set up the spindle contents. This method allows the calling program to reset a puzzle object back to the beginning state. The `reset()` method loops through all three spindles, creating a stack that can hold all of the disks on each one (the `SimpleStack`

implementation uses an array that requires a maximum size). For the first spindle (at index 0), it pushes the integer diameters of the disks on to the stack, starting with the biggest disk first.

The next methods are accessor functions for key values of the spindles. The `label()` method takes a spindle index and returns the label for the spindle in the puzzle. The spindle indices are 0, 1, and 2 for the three spindles. Similarly, the `height()` method takes a spindle index and returns the number of disks on that spindle by returning the length of the stack representing its contents. The `topDisk()` method also takes a spindle index and returns the diameter of the topmost disk on it. If the spindle is empty, it returns `None`.

The `__str__()` method produces a string to show the puzzle state. It puts each spindle on a different line of text showing the spindle label and the stack of disk diameters. For example, the starting state of the three-disk puzzle looks like this:

```
>>> print(TowerOfHanoi(3))
L: [3, 2, 1]
M: []
R: []
```

The bottom of the stack/spindle is on the left of each line. This uses the `SimpleStack` object's `__str__()` method to show the spindle contents.

The `move()` method in [Listing 6-3](#) handles the movement of disks. It enforces the rules of the puzzle and throws exceptions if they are violated. The parameters are the source and destination spindle numbers (`source` and `to`) and a flag (`show`) indicating whether to print the movement information. The first rule to check is that the source spindle is not empty. If the source spindle is empty, it throws an exception. The second rule checks that the destination spindle is either empty or has a disk larger than the topmost one on the source spindle. In other words, if the destination spindle is not empty and has a top disk smaller than that of the source spindle, it throws a different exception. If both rule checks are satisfied, it performs the move by popping the top disk from the source and pushing it onto the destination stack. Finally, if the `show` flag is set, it prints the move information showing the source and destination spindle labels and the disk diameter.

Listing 6-3 *The `TowerOfHanoi.py` Module—Puzzle Movement and Solution*

```
class TowerOfHanoi(object):
```

```
... (other definitions shown before) ...
```

```
    def move(self, source, to,      # Move a single disk from source
              show=False):         # spindle to another, possibly printing
    if self.__stacks[source].isEmpty(): # Source spindle must have
        raise Exception(           # a disk, or it's an error
            "Cannot move from empty spindle " + self.label(source))
    if (not self.__stacks[to].isEmpty() and # Destination cannot
        self.topDisk(source) > # have a disk smaller than that of
        self.topDisk(to)):      # source
        raise Exception(
            "Cannot move disk " + str(self.topDisk(source)) +
            "on top of disk " + str(self.topDisk(to)))
    self.__stacks[to].push( # Push top disk of source spindle
        self.__stacks[source].pop()) # on to the 'to' spindle
    if show:
        print('Move disk', self.topDisk(to),
              'from spindle', self.label(source),
              'to', self.label(to))

    def solve(self,                # Solve the puzzle to move
              nDisks=None,         # N disks from
              start=0,             # starting spindle
              goal=2,              # to goal spindle
              spare=1,             # with spare spindle
              show=False):         # and possibly showing steps
    if nDisks is None:             # Default number of disks to move
        nDisks = self.height(start) # is all the disks on start
    if nDisks <= 0:                # If no request to move disks
        return                     # there's nothing to do
    if self.height(start) < nDisks: # Check if there are fewer
        raise Exception(           # disks to move than requested
            "Not enough disks (" + str(nDisks) +
            ") on starting spindle " + self.label(start))

    self.solve(nDisks - 1, # Move n - 1 from start to spare with
              start, spare, goal, show) # goal as spare
    self.move(start, goal, show) # Move nth from start to goal
    if show: print(self)         # Show puzzle state after move
    self.solve(nDisks - 1, # Then move n - 1 from spare to goal
              spare, goal, start, show) # with start as spare
    if (nDisks == self.__nDisks and # Were all disks moved?
        show):                     # then puzzle is solved and can show
        print("Puzzle complete") # conclusion if requested
```

Finally, the recursive `solve()` method in [Listing 6-3](#) handles all of the solutions to the puzzle. The `solve` method can take all default arguments to solve the full puzzle, or it can take all the parameters needed to solve the "subpuzzles" in the recursive steps. In particular, it needs an `nDisks` parameter specifying how many disks are in the pyramid to be moved. This parameter defaults to `None` and is filled in by the first `if` statement with the number of disks on the starting spindle if no value is provided by the caller. The `start`, `goal`, and `spare` parameters are the indices for the starting, goal, and spare spindles. Because the `reset()` method puts all the disks on the first spindle, index 0, label `L`, they default to 0, 2, and 1, respectively. The final parameter is `show`, a flag to indicate whether to display the moves and intermediate states of the puzzle.

The second `if` statement in the `solve()` method checks for the base case. If the number of disks to move is zero (or somehow negative), there is nothing to be done so it simply returns. The third `if` statement verifies that the number of disks to be moved does not exceed the ones stacked on the starting spindle.

The remaining statements in the `solve()` method execute the recursive solution algorithm outlined previously. The first step is to solve the problem of moving $N-1$ disks from the starting spindle to the spare (nongoal) spindle. This is done by reducing the value of `nDisks` and swapping the roles of the spare and goal spindles. The next statement calls the `move()` method to move the N th disk, which is now on top of the starting spindle, to the goal spindle. If the `show` flag is set, it then prints the state of the puzzle after the move. The next step is the recursive solution to moving $N-1$ disks from the spare spindle on to the goal spindle using the original starting spindle as the spare. The final `if` statement checks whether the puzzle has been solved and prints a message if the `show` flag is set.

You can test the solution by creating and solving puzzles of various sizes, as shown here:

```
>>> TowerOfHanoi(3).solve(show=True)
Move disk 1 from spindle L to R
L: [3, 2]
M: []
R: [1]
Move disk 2 from spindle L to M
L: [3]
M: [2]
R: [1]
```

```

Move disk 1 from spindle R to M
L: [3]
M: [2, 1]
R: []
Move disk 3 from spindle L to R
L: []
M: [2, 1]
R: [3]
Move disk 1 from spindle M to L
L: [1]
M: [2]
R: [3]
Move disk 2 from spindle M to R
L: [1]
M: []
R: [3, 2]
Move disk 1 from spindle L to R
L: []
M: []
R: [3, 2, 1]
Puzzle complete

```

If you run the program with more disks, you should find it dependably enumerating all the steps needed to solve bigger problems. You can use the visualization tool as well. It's quite amazing to see what a few recursive lines of code can produce.

Be careful, however, because the number of moves required to solve a puzzle grows quite fast with the number of disks. How fast?

You can easily find the number of moves needed for the puzzles of size 1 to 10. They're shown in [Table 6-2](#).

Table 6-2 *Moves Required to Solve the Tower of Hanoi with N Disks*

N	1	2	3	4	5	6	7	8	9	10
Moves	1	3	7	15	31	63	127	255	511	1,023

If you've worked with binary numbers, it should be clear that it takes $2^N - 1$ moves to solve the N -disk puzzle. This $O(2^N)$ complexity grows even faster

than the sorting algorithms that you saw in [Chapter 3](#), "Simple Sorting," that were $O(N^2)$. For example, 10^2 is 100, but 2^{10} is 1,024. So, if the legend of the monks solving the puzzle with 64 disks is true, they will have to make $2^{64} - 1$ moves to complete it. If they could complete 1 move every 10 seconds on average, that would take over 5.8×10^{12} years. This is not likely to cause the end of the world any time soon.

It's also important to note that in order to find the $2^N - 1$ moves that solve the puzzle, there will be a chain of only $N + 1$ recursive calls at any time during the computation. This is called the **recursive depth** of the algorithm. It corresponds to the number of nested calls like those shown in [Figures 6-5](#), [6-6](#), [6-7](#), and [6-8](#). There are other algorithms where this recursive depth can grow very large and cause an exception when the execution stack runs out of memory.

Sorting with mergesort

Our final example of recursion is the mergesort. It is a much more efficient sorting technique than those you saw in [Chapter 3](#), at least in terms of speed. The mergesort is also fairly easy to implement. It's conceptually easier than quicksort and the Shellsort, which are described in the next chapter.

The downside of the mergesort is that it requires an additional array in memory, equal in size to the one being sorted. If your original array barely fits in memory, the mergesort won't work. If you have enough space, however, it's a good choice.

Merging Two Sorted Arrays

One of computing science's famous pioneers, John von Neumann, invented the mergesort algorithm by considering merging two already-sorted arrays, A and B. Merging them creates a third array, C, that contains all the elements of A and B, arranged in sorted order. We examine this merging process first.

The two sorted arrays don't need to be the same size. Let's say array A has 4 elements and array B has 6. They will be merged into an array C that starts with 10 empty cells. [Figure 6-15](#) shows these arrays just before the last value is copied into array C.

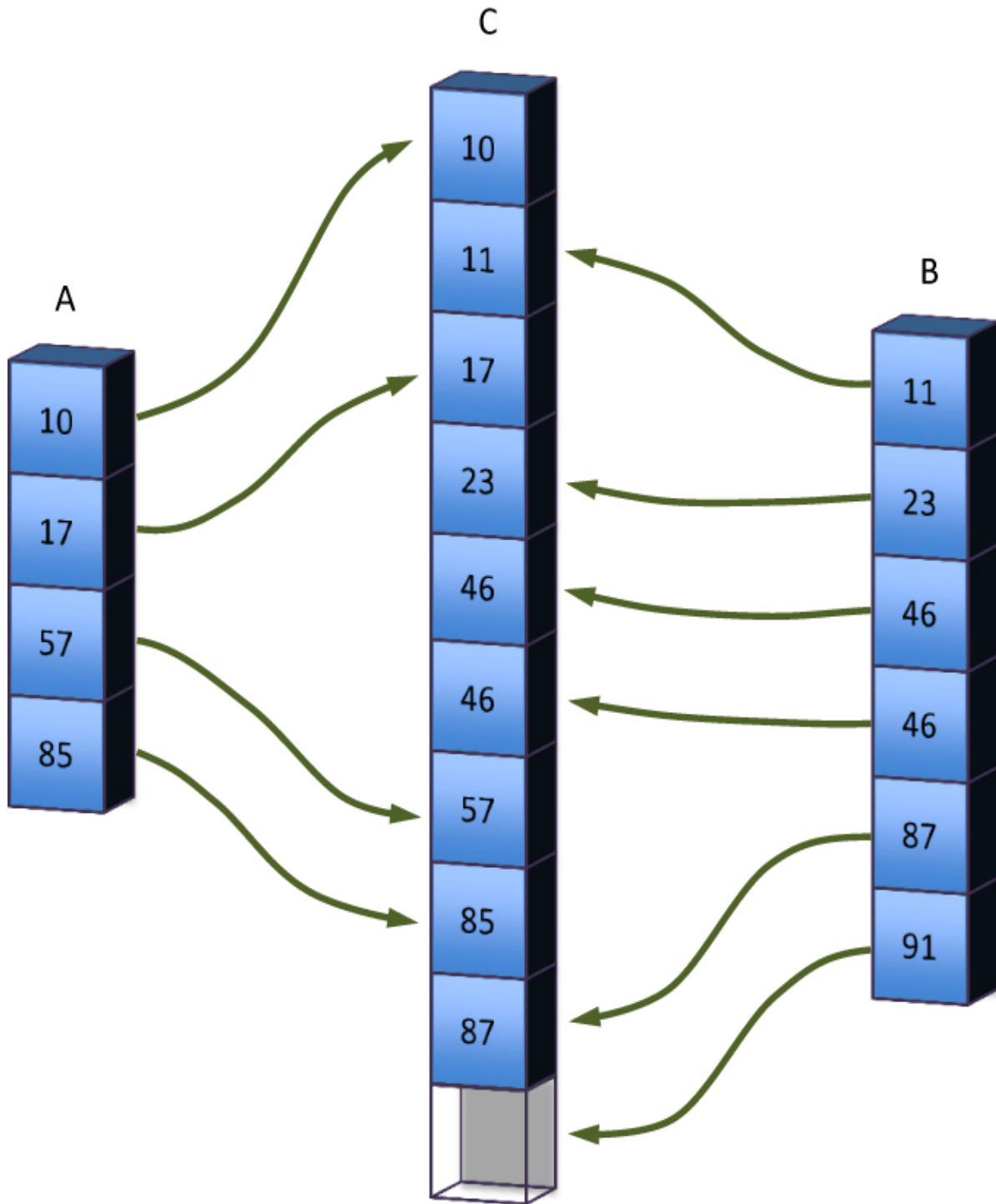


Figure 6-15 *Merging two arrays*

In the figure, the values from arrays A and B are copied to C from top to bottom. Choosing whether to copy from A or B is based on the lowest value remaining to be copied. [Table 6-3](#) shows the comparisons necessary to determine which element will be copied. The steps in the table correspond to

the cells of array C in the figure. Following each comparison, the smaller element is copied to C.

Table 6-3 *Merging Comparisons and Copies*

Step	Comparison (If Any)	Copy
1	Compare 10 and 11	Copy 10 from A to C
2	Compare 17 and 11	Copy 11 from B to C
3	Compare 17 and 23	Copy 17 from A to C
4	Compare 57 and 23	Copy 23 from B to C
5	Compare 57 and 46	Copy 46 from B to C
6	Compare 57 and 46	Copy 46 from B to C
7	Compare 57 and 87	Copy 57 from A to C
8	Compare 85 and 87	Copy 85 from A to C
9		Copy 87 from B to C
10		Copy 91 from B to C

Notice that, because A is empty following step 8, no more comparisons are necessary; all the remaining elements are simply copied from B into C.

Sorting by Merging

The idea in the mergesort is to divide the unsorted, input array in half, sort each half, and then use the merge algorithm just outlined to merge the two halves into a single sorted array. How do you sort each half? This chapter is about recursion, so you probably already know the answer: you divide the half into two quarters, sort each of the quarters, and merge them to make a sorted half.

Similarly, each pair of 8ths is merged to make a sorted quarter, each pair of 16ths is merged to make a sorted 8th, and so on. You divide the array again and again until you reach a subarray with only one element. This is the base case; an array with one element is already sorted.